

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

ОТЧЁТ

Лабораторная работа 3

Контейнеризация микросервисов средствами Docker Compose

Выполнил:
Прель Александр
Группа БР1.1

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Реализовать Dockerfile для каждого сервиса приложения, написать общий docker-compose.yml и настроить сетевое взаимодействие. Проверить локальный запуск и сохранность данных при управлении контейнерами.

Ход работы

1. Исходное приложение и среда

Использованы микросервисы ЛР2 и обмен сообщениями Д35. Бизнес-код остаётся в labs/lab2; новая конфигурация находится в labs/lab3. Монолит labs/lab1 не изменён. Проверка выполнена 10.09.2026 на Mac arm64, Docker Compose 5.5.1. Контейнеры: Linux arm64, Node.js 24.14.0, npm 11.9.0.

Контейнер	Dockerfile / образ	Собственные данные
Accounts	docker/accounts.Dockerfile	accounts_data: users
Catalog	docker/catalog.Dockerfile	catalog_data: каталог, отзывы
Bookings	docker/bookings.Dockerfile	bookings_data: брони, outbox
Notifications	docker/notifications.Dockerfile	notifications_data: уведомления
Gateway	docker/gateway.Dockerfile	Нет базы и тома
RabbitMQ	Официальный образ 4.3.5-management	rabbitmq_data: очереди

2. Multi-stage сборка

В build-стадии установлены Linux-инструменты python3/make/g++, затем выполнены npm ci --include=dev, npm run typecheck и npm run build. npm prune --omit=dev исключает dev-зависимости из runtime. Все пять образов успешно собраны; существующий lockfile и версии приложения не обновлялись.

Runtime использует закреплённый Node.js bookworm-slim, production node_modules, общий код и dist конкретного сервиса. Запуск: node dist/<service>/index.js. Пользователь node, UID 1000. Системные build-инструменты не попадают в финальную стадию и не устанавливаются на Mac.

3. Контекст и конфигурация

.dockerignore разрешает только package.json, package-lock.json, tsconfig.json и src. Секреты, SQLite-файлы и node_modules macOS не передаются в сборку. .env.compose создаётся отдельно с правами 0600, случайными ключами, свободным портом и уникальным именем проекта. Старые .env и базы сохранены. Node.js и RabbitMQ закреплёны тегами и SHA256 digest.

4. Сети и данные

Accounts, Catalog, Bookings, Notifications и RabbitMQ находятся во внутренней backend-сети. Gateway подключён также к edge-сети и публикует только 127.0.0.1:8090 на Mac. Ни API отдельных сервисов, ни AMQP/Management RabbitMQ на host не опубликованы.

Обращение	Адрес внутри Docker
Gateway -> Accounts / Catalog	http://accounts:3000 / http://catalog:3000
Gateway -> Bookings / Notifications	http://bookings:3000 / http://notifications:3000
Проверка JWT	HTTP от защищённого сервиса к Accounts
События бронирований	Bookings -> rabbitmq:5672 -> Notifications

В режиме APP_NETWORK_MODE=compose приложение слушает 0.0.0.0 внутри контейнера и допускает только ожидаемые имена Docker DNS. В прежнем local-режиме сохранена привязка к 127.0.0.1. После пересоздания контейнера его IP может измениться; имя сервиса остаётся прежним.

Каждому владельцу данных назначен отдельный named volume. Базы монтируются в /app/data, брокер - в /var/lib/rabbitmq. У RabbitMQ фиксирован hostname rabbitmq, чтобы сохранить идентичность узла после пересоздания. Данные не хранятся только в удаляемом слое контейнера.

5. Управление и готовность

Из каталога БР1.1/Прель Александр/labs/lab3:

```
npm run setup
npm run config:check
npm run build
npm start
npm run status
```

npm start выполняет compose up --detach --wait и завершается после успешных healthchecks. Контейнеры остаются в Docker после закрытия терминала. Остановка: npm stop. Удаление контейнеров и сетей без томов: npm run down. Повторный запуск: npm start. После изменения кода сначала npm run build. Удалять volumes для обычного перезапуска нельзя.

Node.js-контейнерам заданы read-only root filesystem, отдельный writable volume для БД, tmpfs, cap_drop, no-new-privileges, лимиты памяти/CPU и логов. /health проверяет процесс и БД; /ready Gateway - зависимости и AMQP-соединения. depends_on с service_healthy задаёт порядок запуска; restart: unless-stopped восстанавливает завершившийся процесс, но не перезапускает контейнер только из-за unhealthy.

Первый запуск выявил некорректное экранирование tmpfs-параметров YAML. Затем проверка с Mac обнаружила, что Gateway только во внутренней сети не имеет доступного host-порта. Исправлены кавычки и добавлена edge-сеть только для шлюза. После этого HTTP smoke и Newman успешно прошли.

6. Фактические проверки

Набор	Результат
Локальные тесты lab2	26/26: 17 HTTP, 5 validator, 4 network config
AMQP-регрессия Д35	10/10, реальные broker restart, retries и DLQ
Compose-тесты lab3	8/8, отдельный проект и новые одноразовые тома
Newman на Gateway 8090	29 запросов, 117 assertions, 0 ошибок
HTTP smoke на Gateway 8090	Регистрация, вход, профиль, бронь, отмена, два уведомления

Восемь контейнерных сценариев

1. Шесть контейнеров, внутренняя сеть, только локальный порт Gateway, пять независимых томов, Linux/UID 1000, отсутствие dev-пакетов и .env в образах.
2. Сквозной HTTP, каталог, профиль, отзыв, бронь и приватные уведомления.
3. Двенадцать конкурентных бронирований: один 201 и одиннадцать 409.
4. Реальный SIGKILL Node.js Bookings: Docker перезапускает процесс, бронь сохраняется.
5. Force-recreate пяти сервисов: сохраняются пользователь, отзыв, бронь и уведомление.
6. Остановка RabbitMQ и рестарт Bookings: два события остаются в outbox, затем доставляются.
7. Замена RabbitMQ при остановленном получателе: очередь сохраняется в broker volume.
8. Полный compose down/up без удаления томов: данные остаются доступны.

Тесты используют отдельное случайное имя Compose-проекта, порт и секреты; после прогона удаляют только собственные временные ресурсы. Рабочий стенд и прежние пользовательские базы не очищаются. Проверенный набор сценариев не является доказательством отсутствия всех ошибок.

Вывод

Все три пункта ЛР3 реализованы: подготовлены отдельные Dockerfile сервисов, общий Compose и работающая сеть. API доступен локально на 127.0.0.1:8090. Сохранность данных подтверждена при рестартах и пересоздании контейнеров. Код остаётся в ветке codex/local-setup на базе 922eba1; commit, push и публикация образов не выполнялись.

Ограничения: один Docker Engine, нет TLS, backup/restore, длительной нагрузки и production-миграций. Известные 14 проблем прежнего npm-аудита, включая critical, не исправлены; Linux-образы отдельно на CVE не сканировались. Принятие работ преподавателем не подтверждено.

Материалы и источники

labs/lab3: README.md, docker-compose.yml, TEST_RESULTS.md.

Docker: docs.docker.com/compose/how-tos/networking/;

docs.docker.com/compose/how-tos/startup-order/; docs.docker.com/build/building/best-practices/.